

Statistical Learning 2: Deep Learning

Week 1 Teaching Package – Module 1: Neural Networks as Statistical Models

Assumptions used in this package

- **Module scope:** Module 1 (Week 1, Sessions 1–2) from the course design package: Topic 1.1 (From GLMs to Neural Networks) and Topic 1.2 (Feedforward Networks, Universal Approximation, and Backpropagation).
- **Sessions:** 2 sessions $\times$ 75 minutes.
- **Question type:** Theoretical only (derivations, proofs, conceptual reasoning); the computational side is handled by the separate mini-project sequence.
- **Tool policy:** Pen-and-paper only; no question requires a calculator. The few numbers that appear are chosen to be mentally computable.
- **Topic emphasis:** Not specified, so the question bank leans slightly toward Topic 1.1 (loss functions as negative log-likelihoods). That framing is reused in every later module. The split is 4 questions on 1.1, 2 on 1.2, and 1 bridging both.
- **MOs = Measurable Objectives:** Each Learning Objective (LO) is paired with an MO stating a *condition*, an observable *behavior*, a *criterion*, and the *evidence* source. The numeric criteria (e.g. “4 of 5”) are suggestions to adjust.
- **Evidence timing:** The course plan has no Week 1 quiz. Week 1 MOs are therefore measured by the Week 2 quiz (which covers Week 1 content), by in-class checks, and by Project 1 where relevant.
- **Lesson plan:** A minute-by-minute plan for each session is included (Section ??). This goes beyond the standard breakdown template because you asked for it. Session 1 opens with an 8-minute course orientation; reallocate that time if orientation happens elsewhere.

1 Module Overview and Topic Sequence

Module goal: Frame feedforward networks as nonlinear statistical models $p_{\theta}(y \mid x)$, connect every standard loss function to maximum likelihood, and show that backpropagation is the multivariate chain rule, organized efficiently.

Session	Topic	Arc
1	1.1 From GLMs to Neural Networks	GLM recap $\rightarrow$ loss = negative log-likelihood $\rightarrow$ output layer as a choice of distribution $\rightarrow$ the “prediction minus target” gradient for canonical links $\rightarrow$ a network as a GLM on learned features.
2	1.2 Feedforward Networks, Universal Approximation, Backpropagation	Expressivity (UAT) and what it does <i>not</i> promise $\rightarrow$ backprop by hand for a 2-layer net $\rightarrow$ the same derivation as a computational graph $\rightarrow$ fan-out, skip paths, and why reverse mode is cheap.

2 Lesson Plan

Session 1 (75 min): From GLMs to Neural Networks

Min	Segment	What happens	LOs
0–8	Orientation	Course arc and the “statistical motivation first” rule; projects and weekly quizzes (first quiz in Week 2 covers today and Session 2).	–
8–20	GLM recap	Linear predictor $\eta = w^\top x + b$, mean function, exponential-family response. Logistic regression written as $p_\theta(y x)$ and fit by MLE. Board: Visual aid 1.1-A.	1.1.1
20–35	Loss = NLL	Worked Example 1.1-E1 (Gaussian $\rightarrow$ MSE, plus $\hat{\sigma}^2$). Then Bernoulli $\rightarrow$ BCE stated in parallel. Emphasize that these losses are exact MLE, not heuristics.	1.1.2
35–45	Output layer as distribution	Build the table: identity/Gaussian, sigmoid/Bernoulli, softmax/categorical, exp/Poisson. Worked Example 1.1-E3 (Poisson) done quickly as a class exercise.	1.1.1, 1.1.4
45–57	The $\mu - y$ gradient	Worked Example 1.1-E2 (softmax-CE gradient = $p - y$). Reveal that the Gaussian, Bernoulli, and Poisson cases give the same form: a canonical-link property. Board: Visual aid 1.1-B (MSE vs. BCE on a sigmoid).	1.1.3
57–67	Stacking GLMs	One neuron = one GLM. Hidden layer = learned basis expansion $\phi_\theta(x)$. Network = GLM on $\phi_\theta(x)$. Board: Visual aid 1.1-C.	1.1.1
67–75	Check & preview	Think-pair-share: “Why is squared error on one-hot labels a poor idea?” (Difficult Concept D1). Preview: to fit ϕ_θ we need gradients through the whole stack; that is next session.	1.1.4

Session 2 (75 min): Feedforward Networks, UAT, and Backpropagation

Min	Segment	What happens	LOs
0–5	Recap	The network defines $p_{\theta}(y \mid x)$; training is MLE; we need ∇_{θ} of the NLL.	–
5–20	Universal approximation	Precise statement (key-definition box). Constructive intuition: ReLU “hat” functions (Visual aid 1.2-A). Then Worked Example 1.2-E3: approximation vs. estimation vs. optimization error.	1.2.1, 1.2.2
20–42	Backprop by hand	Worked Example 1.2-E1 on the board, with the shape of every term written underneath. Introduce the error signal $\delta^{(l)}$ only after the chain rule has been written out in full once.	1.2.3
42–55	Same thing, as a graph	Redraw the same network as a computational graph (Visual aid 1.2-B). Show that each backward arrow is a local derivative and that the result matches the hand derivation exactly.	1.2.3, 1.2.5
55–67	Fan-out and skip paths	Worked Example 1.2-E2 (scalar skip connection). Gradients <i>sum</i> over paths (Visual aid 1.2-C). Note the “1+” term as a preview of ResNets (Module 5).	1.2.4
67–75	Why reverse mode; wrap-up	Cost argument: one backward pass for a scalar loss. Separate backprop (computes ∇) from gradient descent (uses ∇). Preview Module 2 and the Project 1 kickoff.	1.2.5

3 Module 1: Neural Networks as Statistical Models

3.1 Topic 1.1: From GLMs to Neural Networks

Concepts

- A GLM has three parts: a linear predictor $\eta = w^\top x + b$, an exponential-family response distribution, and a link g with $g(\mu) = \eta$ (equivalently a mean function $\mu = g^{-1}(\eta)$).
- A neural network is a conditional model $p_\theta(y \mid x)$: the final layer produces the natural parameter(s) of an assumed output distribution, exactly like a GLM, but on learned features $\phi_\theta(x)$ instead of raw x .
- Training by minimizing a loss is maximum likelihood: $\hat{\theta} = \arg \min_\theta \sum_i -\log p_\theta(y_i \mid x_i)$.
- Gaussian noise $\leftrightarrow$ squared error; Bernoulli $\leftrightarrow$ binary cross-entropy; categorical $\leftrightarrow$ (softmax) cross-entropy; Poisson $\leftrightarrow$ Poisson NLL $e^f - yf$.
- The final-layer activation is a modeling choice. Identity, sigmoid, softmax, and exp are the inverse canonical links of the Gaussian, Bernoulli, categorical, and Poisson families.
- With a canonical link, the gradient of the NLL with respect to the pre-activation is always $\mu - y$ (prediction minus target).
- Softmax is invariant to adding a constant to all logits: K logits carry only $K - 1$ degrees of freedom.

Key Definitions – Topic 1.1

Conditional model.

“A neural network for supervised learning is a family of conditional distributions $p_\theta(y \mid x)$. The network’s output is not the prediction itself; it is the *parameter* of a distribution over y .”

Canonical link.

“The link function g for which the linear predictor equals the natural parameter of the exponential family: $\eta = g(\mu)$ is the natural parameter. Logit for Bernoulli, log for Poisson, identity for Gaussian.”

Negative log-likelihood (NLL).

“ $\mathcal{L}(\theta) = -\sum_{i=1}^n \log p_\theta(y_i \mid x_i)$. Minimizing it over θ is maximum likelihood estimation. Every standard loss in this course is an NLL under some noise model, possibly plus a penalty.”

Logits.

“The unnormalized real-valued outputs $z \in \mathbb{R}^K$ of the final layer, before softmax. They are not probabilities, and they are identified only up to an additive constant.”

Softmax.

“ $\text{softmax}(z)_k = e^{z_k} / \sum_{j=1}^K e^{z_j}$. It maps $\mathbb{R}^K$ onto the interior of the probability simplex and is the inverse canonical link of the categorical distribution.”

Cross-entropy.

“For a target distribution q and model distribution p : $H(q, p) = -\sum_k q_k \log p_k$. With a one-hot target $q = y$, this equals the categorical NLL of that example. ‘Cross-entropy loss’ and ‘categorical NLL’ are the same number.”

Learning Objectives

- LO 1.1.1 Explain** a feedforward network as a conditional model $p_{\theta}(y \mid x)$ whose output layer parameterizes an exponential-family distribution, by analogy with a GLM.
- LO 1.1.2 Derive** the loss function implied by a stated noise model (Gaussian, Laplace, Bernoulli, categorical, Poisson).
- LO 1.1.3 Derive** the gradient of the NLL with respect to the output pre-activation for canonical links, and **recognize** the common form $\mu - y$.
- LO 1.1.4 Evaluate** whether a proposed output-activation/loss pair is statistically coherent for a given target type, and **justify** a correct replacement.

Measurable Objectives

MO	Condition	Behavior	Criterion	Evidence
1.1.1	Given a target type (real, binary, K -class, count)	Writes $p_{\theta}(y \mid x)$ explicitly, naming the distribution and which network output is its parameter	Correct for 4 of 4 target types	Week 2 quiz; in-class check
1.1.2	Given a stated conditional noise model, closed book	Writes the per-example NLL and reduces it to the named loss, keeping all θ -dependent terms and discarding only constants	Correct reduction for ≥ 4 of 5 models; no dropped θ -dependent terms	Week 2 quiz
1.1.3	Given softmax, sigmoid, or exp output with its matching NLL	Derives $\partial \mathcal{L} / \partial z$ and states it as $\mu - y$	Fully correct derivation for ≥ 2 of 3 cases, including the softmax case	Week 2 quiz
1.1.4	Given a mismatched pair (e.g. MSE on sigmoid for classification)	Identifies the implicit noise assumption, names one concrete consequence (e.g. gradient saturation), and proposes the coherent pair	All three components present	In-class think-pair-share; Week 2 quiz

Prerequisites

- Maximum likelihood estimation; log-likelihood of i.i.d. data. *Assumed background (Probability & Inference).*
- Linear and logistic regression as GLMs; exponential-family basics. *Assumed background (Statistical Learning 1).*
- Gradients of scalar functions of vectors. *Assumed background (Multivariate Calculus).*

Difficult Concepts

D1. “MSE and cross-entropy are interchangeable, just a matter of taste.”

Misconception: students treat the loss as a free hyperparameter disconnected from any model of the data. With a sigmoid output, MSE gives $\partial \mathcal{L} / \partial z = (p - y)p(1 - p)$. This vanishes when the model is confidently wrong ($p \rightarrow 0$ with $y = 1$), so the worst errors produce the smallest gradients.

Mitigation: derive both gradients side by side on the board, then evaluate each at $z = -5$, $y = 1$ ($p \approx 0.007$). MSE gives roughly -0.007 ; BCE gives roughly -0.99 . Pair this with Visual aid 1.1-B.

D2. “The network’s output *is* the prediction.”

Abstraction jump: students must move from $\hat{y} = f(x)$ to $f(x)$ as a *parameter* of $p(y | x)$. Without this move, heteroscedastic outputs, mixture outputs, and later generative models (Modules 10–11) do not make sense. *Mitigation:* insist on the notation $y | x \sim \text{Dist}(g^{-1}(f_\theta(x)))$ in every example this week. Include one example where the network outputs two things, a mean and a log-variance (Bank Q4).

D3. “ σ^2 always drops out, so Gaussian MLE never involves the noise level.”

Misconception: σ^2 drops out of the arg min over θ only when it is fixed and shared across examples. If σ^2 depends on x and is learned, it reweights the residuals and can make the likelihood unbounded.

Mitigation: in Example E1, ask “what if σ depended on x ?” as a closing question, and defer the full answer to Bank Q4.

D4. Logits vs. probabilities.

Misconception: reading raw logits as probabilities or as “confidence scores,” and being surprised that softmax is shift-invariant. *Mitigation:* show on the board that $\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z)$. Then show that binary logistic regression is the $K = 2$ softmax with one logit pinned to 0.

Example Questions (for presentation in class)

1.1-E1 Suppose $y_i | x_i \sim \mathcal{N}(f_\theta(x_i), \sigma^2)$ independently, with σ^2 fixed and unknown. (a) Write the total NLL. (b) Show that $\hat{\theta}$ minimizes squared error regardless of σ^2 . (c) Find the MLE of σ^2 .

Solution. (a) $-\log p = \sum_{i=1}^n \left[\frac{1}{2} \log(2\pi\sigma^2) + \frac{(y_i - f_\theta(x_i))^2}{2\sigma^2} \right] = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_i r_i^2$,

where $r_i = y_i - f_\theta(x_i)$. (b) For any fixed $\sigma^2 > 0$, the first term does not depend on θ and the second is a positive multiple of $\sum_i r_i^2$. The minimizer over θ is therefore the least-squares solution, and it is the same for every σ^2 . (c) Substitute $\hat{\theta}$ and set $S = \sum_i \hat{r}_i^2$. Then $\partial / \partial \sigma^2 [\frac{n}{2} \log \sigma^2 + S/(2\sigma^2)] = \frac{n}{2\sigma^2} - \frac{S}{2\sigma^4} = 0$, which gives $\hat{\sigma}^2 = \frac{1}{n} \sum_i \hat{r}_i^2$. This is the mean squared training residual. It is biased downward, since it divides by n and not by n minus the effective number of parameters, and for an overparameterized network it can be near 0. That point is a preview of Module 3.

1.1-E2 Let $p = \text{softmax}(z)$ with $z \in \mathbb{R}^K$ and one-hot target y . Show that $\partial \mathcal{L} / \partial z = p - y$ for $\mathcal{L} = -\sum_k y_k \log p_k$.

Solution. $\log p_k = z_k - \log \sum_j e^{z_j}$. Therefore $\mathcal{L} = -\sum_k y_k z_k + (\sum_k y_k) \log \sum_j e^{z_j} = -\sum_k y_k z_k + \log \sum_j e^{z_j}$, using $\sum_k y_k = 1$. Differentiating with respect to z_m : $\partial \mathcal{L} / \partial z_m = -y_m + e^{z_m} / \sum_j e^{z_j} = p_m - y_m$. *Teaching point:* this is the canonical-link result. Gaussian with identity link gives $f - y$, Bernoulli with sigmoid gives $p - y$, and Poisson with exp gives $\lambda - y$ (E3). The error signal that starts backprop is always “prediction minus target.”

1.1-E3 You are predicting daily counts $y \in \{0, 1, 2, \dots\}$. Choose an output distribution, an output activation, and a loss, and compute $\partial\mathcal{L}/\partial f$ where f is the pre-activation.

Solution. Use $y \mid x \sim \text{Poisson}(\lambda(x))$ with $\lambda = e^{f_{\theta}(x)}$. The exp activation guarantees $\lambda > 0$ and is the inverse canonical (log) link. Then $-\log p = \lambda - y \log \lambda + \log y! = e^f - yf + \text{const}$, and $\partial\mathcal{L}/\partial f = e^f - y = \lambda - y$. *Contrast:* a ReLU output with MSE would allow $\lambda = 0$ exactly (log-likelihood $-\infty$ whenever $y > 0$). It would also assume constant variance, whereas Poisson counts have variance equal to their mean.

Instructor note – Visual aids – Topic 1.1

1.1-A: Regression as conditional density.

Horizontal axis x , vertical axis y . Draw a scatter of about 15 points around a smooth increasing curve labeled $f_{\theta}(x)$. At three x locations (x_1, x_2, x_3) , draw a small Gaussian bell turned sideways, centered on the curve and extending along the y direction, all of equal width. Label one width σ . Caption on the board: “the model is the whole family of bells, not the curve.” Optional second panel: the same picture with bells widening as x increases, labeled “heteroscedastic: $\sigma(x)$.”

1.1-B: MSE vs. BCE on a sigmoid output, target $y = 1$.

Horizontal axis: $\text{logit } z$ from -6 to 6 . Draw two loss curves. BCE, $-\log \sigma(z)$, is roughly a straight line of slope -1 on the far left, bends near $z = 0$, and flattens toward 0 on the right. MSE, $\frac{1}{2}(1 - \sigma(z))^2$, is an S-shape that plateaus near 0.5 on the left, meaning it is flat exactly where the model is badly wrong, and approaches 0 on the right. Mark $z = -5$ on both curves and draw short tangent segments: steep on BCE, nearly horizontal on MSE. Label the flat region “confidently wrong, no gradient.”

1.1-C: From one GLM to a network.

Left: a single node receiving arrows from $x_1, \dots, x_d$, labeled “ $\eta = w^\top x + b$, then g^{-1} ” (logistic regression). Right: the same output node, but now its inputs come from a column of hidden nodes $h_1, \dots, h_m$, which themselves receive $x_1, \dots, x_d$. Bracket the hidden column as “learned features $\phi_{\theta}(x)$ ” and the output node as “a GLM on $\phi_{\theta}(x)$.”

Output-layer table (write as a 4-row board table).

Columns: target type — distribution — output activation (inverse canonical link) — loss — $\partial\mathcal{L}/\partial z$. Rows: real — Gaussian — identity — squared error — $f - y$. Binary — Bernoulli — sigmoid — BCE — $p - y$. K -class — categorical — softmax — CE — $p - y$. Count — Poisson — exp — $e^f - yf$ — $\lambda - y$. Circle the last column.

3.2 Topic 1.2: Feedforward Networks, Universal Approximation, and Back-propagation

Concepts

- An L -layer feedforward network alternates affine maps and elementwise nonlinearities: $a^{(0)} = x$, $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$, $a^{(l)} = \sigma(z^{(l)})$, with the final layer feeding the output distribution from Topic 1.1.
- Universal approximation is a statement about *expressivity* only. Expressivity, trainability (optimization), and generalization (estimation) are three separate questions.

- ReLU networks compute continuous piecewise-linear functions. Constructing such functions explicitly makes the UAT plausible.
- Backpropagation is the multivariate chain rule applied in reverse order on a computational graph, reusing the intermediate quantities from the forward pass.
- Layer recursion: $\delta^{(L)} = \partial\mathcal{L}/\partial z^{(L)}$; $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \sigma'(z^{(l)})$; $\partial\mathcal{L}/\partial W^{(l)} = \delta^{(l)} a^{(l-1)\top}$; $\partial\mathcal{L}/\partial b^{(l)} = \delta^{(l)}$.
- At a fan-out node, the gradients arriving from different downstream paths *add*.
- For a scalar loss, reverse mode obtains the entire gradient in one backward pass, at a small constant multiple of the forward cost. Forward mode would need one pass per parameter.

Key Definitions – Topic 1.2

Universal approximation theorem (one hidden layer).

“Let σ be continuous and not a polynomial. For every compact $K \subset \mathbb{R}^d$, every continuous $g : K \rightarrow \mathbb{R}$, and every $\varepsilon > 0$, there exist a width m and parameters such that $\sup_{x \in K} |g(x) - \sum_{j=1}^m c_j \sigma(w_j^\top x + b_j)| < \varepsilon$.” State all three limits out loud: the width m may be enormous; nothing is said outside K ; and nothing is said about whether training on finite data finds these weights.

Computational graph.

“A directed acyclic graph whose nodes are intermediate quantities and whose edges record which quantities each node is computed from. Each edge carries a *local* derivative.”

Backpropagation.

“Reverse-mode automatic differentiation applied to a scalar loss: after one forward pass, traverse the graph from output to inputs, multiplying each node’s upstream gradient by local derivatives and summing over outgoing paths. It *computes* gradients. It is not an optimization method.”

Error signal $\delta^{(l)}$.

“ $\delta^{(l)} = \partial\mathcal{L}/\partial z^{(l)}$, the gradient of the loss with respect to layer l ’s *pre*-activation. It has the same shape as $z^{(l)}$.”

Shape convention.

“For a scalar $\mathcal{L}$, $\partial\mathcal{L}/\partial W$ has the same shape as W . Every gradient we write must pass this shape check.”

Learning Objectives

- LO 1.2.1 State** the universal approximation theorem precisely and **explain** what it does not guarantee, in terms of approximation, estimation, and optimization error.
- LO 1.2.2 Construct** small ReLU networks that exactly represent given piecewise-linear functions.
- LO 1.2.3 Derive** backpropagation for a two-layer network in matrix form, with correct shapes and transposes.
- LO 1.2.4 Apply** the chain rule on a computational graph containing fan-out or skip connections.
- LO 1.2.5 Explain** why reverse-mode differentiation is efficient for scalar losses, and **distinguish** backpropagation from gradient descent.

Measurable Objectives

MO	Condition	Behavior	Criterion	Evidence
1.2.1	Given a claim that “UAT guarantees” some learning outcome	States the theorem’s hypotheses and conclusion, and names which error type(s) the claim ignores	All three limits identified (width, domain, learnability)	Week 2 quiz
1.2.2	Given a continuous piecewise-linear function with ≤ 3 breakpoints	Writes a ReLU network (weights, biases) that represents it exactly	Exact representation; verified on every linear piece	Week 2 quiz
1.2.3	Given $f(x) = W_2\sigma(W_1x + b_1) + b_2$ and a loss, closed book	Writes the gradients w.r.t. W_1, b_1, W_2, b_2 with the dimension of every factor	All four correct, including transposes and $\odot$ placement	Week 2 quiz; Project 1 (implementation must pass a gradient check)
1.2.4	Given a scalar graph with one fan-out or skip path	Computes the requested partial derivative by explicitly summing over paths	Correct value, with each path’s contribution shown separately	In-class check; Week 2 quiz
1.2.5	Given $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$	States the number of passes forward mode and reverse mode need for the full Jacobian, and applies this to $m = 1$	Correct counts and a correct conclusion	Week 2 quiz

Prerequisites

- Multivariate chain rule; Jacobians; matrix–vector products and transposes. *Assumed background (Linear Algebra, Multivariate Calculus).*
- Networks as conditional models; the $\mu - y$ output error signal. *Earlier in this module (Topic 1.1).*
- Uniform (sup-norm) approximation of continuous functions on compact sets. *Assumed background (Analysis). Restated briefly in class.*

Difficult Concepts

D5. “The UAT explains why deep learning works.”

Misconception: the theorem is about *one* hidden layer and an existence statement, so it says nothing about depth, sample size, or SGD. Students also read it as covering extrapolation, when it only concerns a compact set. *Mitigation:* write the excess-risk decomposition (approximation + estimation + optimization) on the board. Put a check mark under “approximation” and a question mark under the other two, then label the question marks “Module 2” and “Module 3.”

D6. Transpose and shape errors in matrix backprop.

Misconception: writing $\partial\mathcal{L}/\partial W_1 = x\delta_1^\top$ or $W_2\delta_2$ instead of $W_2^\top\delta_2$. These errors come from pattern-matching scalar backprop onto matrices. *Mitigation:* a shape discipline that is required on every

board derivation and quiz. Write the dimensions under every factor $(\underbrace{\delta_1}_{h \times 1} \underbrace{x^\top}_{1 \times d})$, and reject any expression whose shape differs from W 's.

D7. Fan-out: multiply or add?

Misconception: students multiply the gradients from two downstream paths, confusing “chain along a path” (multiply) with “combine across paths” (add). *Mitigation:* use the scalar skip example E2 with numbers. Have students compute both paths separately, then check the sum against direct differentiation of the closed-form expression.

D8. “Backprop” = “training.”

Misconception: students treat backprop as the learning algorithm. This later causes confusion when the same gradients feed Adam, when gradients are taken with respect to inputs (adversarial examples), or in second-order methods. *Mitigation:* write the training loop as two lines, $g \leftarrow \text{backprop}(\theta)$ and $\theta \leftarrow \text{update}(\theta, g)$, and point out that Module 2 changes only the second line.

Example Questions (for presentation in class)

1.2-E1 Let $x \in \mathbb{R}^d$, $z_1 = W_1 x + b_1 \in \mathbb{R}^h$, $a_1 = \sigma(z_1)$, $f = W_2 a_1 + b_2 \in \mathbb{R}^m$, and $\mathcal{L} = \frac{1}{2} \|f - y\|^2$. Derive all four parameter gradients and check their shapes.

Solution. Output error: $\delta_2 = \partial \mathcal{L} / \partial f = f - y \in \mathbb{R}^m$. This is the Gaussian case of $\mu - y$. Since $f_i = \sum_j (W_2)_{ij} (a_1)_j + (b_2)_i$: $\partial \mathcal{L} / \partial (W_2)_{ij} = (\delta_2)_i (a_1)_j$, so $\partial \mathcal{L} / \partial W_2 = \delta_2 a_1^\top$ ($m \times h$, the same shape as W_2). Also $\partial \mathcal{L} / \partial b_2 = \delta_2$. Back into the hidden layer: $\partial \mathcal{L} / \partial (a_1)_j = \sum_i (\delta_2)_i (W_2)_{ij}$, so $\partial \mathcal{L} / \partial a_1 = W_2^\top \delta_2 \in \mathbb{R}^h$. Through the elementwise nonlinearity: $\delta_1 = (W_2^\top \delta_2) \odot \sigma'(z_1)$. Then $\partial \mathcal{L} / \partial W_1 = \delta_1 x^\top$ ($h \times d$) and $\partial \mathcal{L} / \partial b_1 = \delta_1$. For a minibatch, stack examples as rows ($X \in \mathbb{R}^{n \times d}$, $\Delta_1 \in \mathbb{R}^{n \times h}$) and sum over examples: $\partial \mathcal{L} / \partial W_1 = \Delta_1^\top X$.

1.2-E2 Scalar skip connection: $u = \text{ReLU}(wx)$, $o = u + x$, $\mathcal{L} = \frac{1}{2} (o - t)^2$. (a) Find $\partial \mathcal{L} / \partial x$ and $\partial \mathcal{L} / \partial w$ symbolically. (b) Evaluate at $x = 2$, $w = 1.5$, $t = 4$.

Solution. (a) $\partial \mathcal{L} / \partial o = o - t$. The variable x reaches o along two paths: directly, with local derivative 1, and through u , with local derivative $\partial u / \partial x = w \mathbf{1}[wx > 0]$. The contributions add: $\partial \mathcal{L} / \partial x = (o - t)(1 + w \mathbf{1}[wx > 0])$. The weight w reaches o only through u : $\partial \mathcal{L} / \partial w = (o - t)x \mathbf{1}[wx > 0]$. (b) $wx = 3 > 0$, so $u = 3$, $o = 5$, and $o - t = 1$. This gives $\partial \mathcal{L} / \partial x = 1 \cdot (1 + 1.5) = 2.5$ and $\partial \mathcal{L} / \partial w = 1 \cdot 2 \cdot 1 = 2$. Check directly: on this region $o = (1 + w)x$, so $\partial o / \partial x = 1 + w = 2.5$. *Teaching point:* the “1+” guarantees a gradient path even when the ReLU is off ($wx < 0$). This is the core idea of residual networks (Module 5).

1.2-E3 A classmate says: “By the UAT, a one-hidden-layer network trained with SGD on 1000 examples will learn any continuous regression function.” Identify every gap in this argument.

Solution. Decompose the excess risk of the trained network $\hat{f}$ into three parts. *Approximation error:* the UAT says it can be made small, but only with a width that may be huge. For generic continuous or Lipschitz targets the required width can grow exponentially in d , and the theorem gives no rate. *Estimation error:* with 1000 examples and many parameters, the best-in-class function on the *training* data need not be close to the target. The UAT says nothing about sample size. *Optimization error:* the objective is nonconvex, and SGD may not reach the approximating weights, which the UAT only asserts *exist*. Also, the guarantee holds only on a compact set K , with no claim about inputs

outside the training domain. A full-credit answer names the approximation, estimation, and optimization gaps; the domain restriction is a bonus.

Instructor note – Visual aids – Topic 1.2

1.2-A: Building a bump from ReLUs.

Use three small stacked panels with a shared horizontal axis x from -1 to 3 . Panel 1: $\text{ReLU}(x)$, flat at 0 then slope 1 from $x = 0$. Panel 2: $-2 \text{ReLU}(x - 1)$, flat at 0 then slope -2 from $x = 1$. Panel 3: $\text{ReLU}(x - 2)$, slope 1 from $x = 2$. Below them, the sum: a triangular “hat” that is 0 for $x \leq 0$, rises to height 1 at $x = 1$, falls to 0 at $x = 2$, and stays 0 after. Mark the breakpoints 0, 1, 2 with dotted vertical lines through all panels. Then sketch an arbitrary smooth curve on $[0, 4]$ with a sequence of adjacent hats of varying heights underneath whose sum traces a piecewise-linear interpolant of the curve. Label it “one hidden unit per breakpoint.”

1.2-B: Computational graph of the 2-layer net (E1).

Place boxes left to right: $x, z_1, a_1, f, \mathcal{L}$, with parameter boxes W_1, b_1 feeding z_1 from above, W_2, b_2 feeding f from above, and y feeding $\mathcal{L}$ from below. Draw forward arrows in one color. Under each forward arrow, draw a backward arrow in a second color labeled with its local derivative: $\mathcal{L} \rightarrow f: (f - y)$; $f \rightarrow a_1: W_2^\top$; $a_1 \rightarrow z_1: \odot \sigma'(z_1)$; $f \rightarrow W_2: \cdot a_1^\top$; $z_1 \rightarrow W_1: \cdot x^\top$. Write the shape next to every box.

1.2-C: Fan-out and skip path (E2).

Draw a node x with two outgoing arrows. The upper arrow goes to a node wx , then to ReLU, then to u . The lower arrow goes straight to a “+” node. The node u also feeds the “+” node, whose output is o , which feeds $\mathcal{L}$. In the backward color, label the upper route $w \mathbf{1}[wx > 0]$ and the lower route 1. At x , draw the two backward arrows merging into a circled “+” with the caption “paths add.”

4 Module Question Bank

Theoretical, pen-and-paper questions, tagged by topic and difficulty. Weighted slightly toward Topic 1.1 (see Assumptions). No question needs a provided diagram. Q5 asks students to produce a sketch themselves.

Q1 [Topic 1.1 — Basic] Suppose $y \mid x \sim \text{Laplace}(f_\theta(x), b)$, with density $\frac{1}{2b} \exp(-|y - f_\theta(x)|/b)$ and b fixed. Derive the loss implied by maximum likelihood. Which conditional summary of y does its population minimizer estimate?

Solution. $-\log p = \log(2b) + |y - f_\theta(x)|/b$. Dropping the constant and the positive scale $1/b$, MLE minimizes $\sum_i |y_i - f_\theta(x_i)|$, the absolute-error (L1) loss. For each x , $\mathbb{E}|Y - c|$ is minimized at $c = \text{median}(Y \mid x)$: the subgradient is $P(Y < c \mid x) - P(Y > c \mid x)$, which is zero at the median. So f_θ targets the conditional median. By comparison, squared error (Gaussian noise) targets the conditional mean. Choosing the noise model chooses the summary being estimated, and the heavier Laplace tails explain L1’s robustness to outliers.

Q2 [Topic 1.1 — Basic] (a) Show that $\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z)$ for any scalar c . (b) Use this to show that binary logistic regression is the $K = 2$ softmax model, and state how many free logits a K -class model really has.

Solution. (a) $\frac{e^{z_k+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_k}}{e^c \sum_j e^{z_j}} = \text{softmax}(z)_k$. (b) With $K = 2$, choose $c = -z_2$:

$p_1 = \frac{e^{z_1-z_2}}{e^{z_1-z_2} + 1} = \sigma(z_1 - z_2)$. Only the difference $z_1 - z_2$ matters, and it plays the role of the single logistic-regression logit. In general, the K logits are identified only up to a common shift, so there are $K - 1$ free logits. This is why classical multinomial regression pins one class as a reference; in neural nets the redundancy is harmless and usually left in.

Q3 [Topic 1.1 — Intermediate] A binary classifier has output $p = \sigma(z)$. Compute $\partial \mathcal{L} / \partial z$ under (i) $\mathcal{L}_{\text{MSE}} = \frac{1}{2}(p - y)^2$ and (ii) $\mathcal{L}_{\text{BCE}} = -[y \log p + (1 - y) \log(1 - p)]$. Show that when $y = 1$ and $z \rightarrow -\infty$, one gradient vanishes and the other does not, and explain the statistical reason.

Solution. Use $\sigma'(z) = p(1 - p)$. (i) $\partial \mathcal{L}_{\text{MSE}} / \partial z = (p - y) p(1 - p)$. (ii) $\partial \mathcal{L}_{\text{BCE}} / \partial p = -y/p + (1 - y)/(1 - p)$; multiplying by $p(1 - p)$ gives $-y(1 - p) + (1 - y)p = p - y$. When $y = 1$ and $z \rightarrow -\infty$, $p \rightarrow 0$. Then (i) $\approx (-1) \cdot p \rightarrow 0$ while (ii) $\rightarrow -1$. *Reason:* BCE is the Bernoulli NLL with the canonical (logit) link. The sigmoid's saturation factor $p(1 - p)$ cancels exactly against the likelihood's $1/(p(1 - p))$ curvature, leaving $\mu - y$. MSE imposes a Gaussian noise model on a $\{0, 1\}$ target, so there is no such cancellation, and the most confidently wrong predictions receive almost no corrective signal.

Q4 [Topic 1.1 — Advanced] A regression network outputs two numbers per input: a mean $\mu_\theta(x)$ and $s_\theta(x) = \log \sigma_\theta^2(x)$, with $y \mid x \sim \mathcal{N}(\mu_\theta(x), e^{s_\theta(x)})$. (a) Write the per-example NLL. (b) For a fixed residual $r = y - \mu$, find the minimizing s and the minimized loss. What happens as $r \rightarrow 0$? (c) Compute $\partial \mathcal{L} / \partial \mu$ and explain how the learned variance changes the influence of each data point. Name one practical failure mode that follows, and one remedy.

Solution. (a) $\mathcal{L} = \frac{1}{2} \log(2\pi) + \frac{1}{2}s + \frac{1}{2}r^2 e^{-s}$. (b) $\partial \mathcal{L} / \partial s = \frac{1}{2} - \frac{1}{2}r^2 e^{-s} = 0$ gives $e^s = r^2$, and the minimized loss is $\frac{1}{2} \log(2\pi) + \log|r| + \frac{1}{2}$. As $r \rightarrow 0$ this tends to $-\infty$. A flexible network that interpolates even one training point can drive the likelihood to $+\infty$ by shrinking σ^2 there, so the MLE is degenerate, as with Gaussian mixtures when a component collapses onto one point. (c) $\partial \mathcal{L} / \partial \mu = -(y - \mu) e^{-s} = -r / \sigma^2$. Each point's pull on the mean is scaled by $1/\sigma_\theta^2(x)$. *Failure mode:* the network can “give up” on hard regions by inflating σ^2 there, which shrinks their gradient so the mean fit in those regions stalls. Symmetrically, easy regions get their variance collapsed. *Remedies (any one):* a variance floor $\sigma^2 \geq \sigma_{\min}^2$; first training μ with a fixed variance, then fitting s ; weight decay or other regularization on the variance head; or a reweighted objective that partially cancels the $1/\sigma^2$ scaling. *Rubric:* (a) and (b) are required; (c) requires the $1/\sigma^2$ weighting and one failure mode; the remedy is a bonus.

Q5 [Topic 1.2 — Basic] (a) Sketch $g(x) = \text{ReLU}(x) - 2 \text{ReLU}(x - 1) + \text{ReLU}(x - 2)$ on $[-1, 3]$, giving its formula on each linear piece. (b) Show that any continuous piecewise-linear $h : \mathbb{R} \rightarrow \mathbb{R}$ with breakpoints $t_1 < \dots < t_k$ can be written as $h(x) = \alpha + \beta x + \sum_{j=1}^k c_j \text{ReLU}(x - t_j)$, and say what c_j is. (c) Why does (b) make the universal approximation theorem plausible in one dimension, and what does it *not* tell you about how many units are needed?

Solution. (a) $g = 0$ for $x \leq 0$; $g = x$ on $[0, 1]$; $g = x - 2(x - 1) = 2 - x$ on $[1, 2]$; $g = 2 - x + (x - 2) = 0$ for $x \geq 2$. This is a hat of height 1 at $x = 1$, supported on $[0, 2]$. (b) Take β to be the slope of h left of t_1 and α the matching intercept, so the formula agrees with h for $x < t_1$. Each $\text{ReLU}(x - t_j)$ is zero left of t_j and has slope 1 to the right, so adding $c_j \text{ReLU}(x - t_j)$ changes the slope by exactly c_j at t_j and nowhere else. Set c_j equal to the slope of h just right of t_j minus the slope just left of it. The two continuous functions then agree on the first piece and have equal slopes on every piece, so they are equal everywhere. (c) Any continuous function on $[a, b]$ is uniformly continuous, so its piecewise-linear interpolant

on a fine enough grid is uniformly within ε of it. By (b), that interpolant is a one-hidden-layer ReLU network. *Not answered:* how fine the grid must be. For an L -Lipschitz target, a spacing of order ε/L suffices, so about L/ε units are needed in 1-D. In d dimensions, grid-based constructions need on the order of $(L/\varepsilon)^d$ units. The existence argument gives no efficient rate in general.

- Q6** [Topic 1.2 — Advanced] Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be a composition of differentiable maps. (a) Forward-mode differentiation computes one Jacobian–vector product Jv per pass, and reverse mode computes one vector–Jacobian product $u^\top J$ per pass. How many passes does each need to obtain the full Jacobian $J \in \mathbb{R}^{m \times n}$? (b) Apply this to training a network with $n = 10^8$ parameters and a scalar loss. (c) Give one setting where forward mode is the better choice, and state the main resource cost that reverse mode pays in exchange for its speed.

Solution. (a) Forward mode recovers one *column* of J per pass (take $v = e_j$), so it needs n passes. Reverse mode recovers one *row* per pass (take $u = e_i$), so it needs m passes. (b) With $m = 1$, one reverse pass (backprop) gives the full gradient, at a small constant multiple of the cost of one forward evaluation. Forward mode would need 10^8 passes. This asymmetry is the whole reason gradient-based training of large networks is feasible. (c) Forward mode is better when $n \ll m$. Examples: the sensitivity of many outputs to a single scalar input (such as one hyperparameter or a time variable), or Jacobian–vector products inside some second-order or physics-informed methods. *Cost of reverse mode:* memory. All intermediate activations from the forward pass must be stored (or recomputed, as in checkpointing) for the backward pass, so memory grows with depth and batch size.

- Q7** [Topics 1.1 & 1.2 — Intermediate] A K -class network (any architecture) ends with logits $z_i = V \phi_\theta(x_i) + c$, trained by minimizing the softmax cross-entropy $\sum_{i=1}^n - \sum_k y_{ik} \log p_{ik}$. Show that at *any* stationary point of the training objective, $\frac{1}{n} \sum_i p_{ik} = \frac{1}{n} \sum_i y_{ik}$ for every class k . Interpret this result, and state what it does *not* imply.

Solution. By Example 1.1-E2, $\partial \mathcal{L}_i / \partial z_{ik} = p_{ik} - y_{ik}$. Since z_{ik} depends on c_k with derivative 1, the chain rule gives $\partial \mathcal{L} / \partial c_k = \sum_i (p_{ik} - y_{ik})$. At a stationary point this is zero for every k , which is the claimed identity. (The same argument on V gives $\sum_i (p_i - y_i) \phi_\theta(x_i)^\top = 0$: GLM score equations, with the learned features playing the role of the covariates.) *Interpretation:* whatever the hidden layers do, an unregularized output bias at a stationary point forces the average predicted probability of each class to match its training frequency (“calibration-in-the-large” on the training set). *Not implied:* calibration on test data, calibration within subgroups or confidence bins, or good accuracy. The result holds on the training set only, and it can fail if the bias is regularized or training stops early. Test-set miscalibration of modern networks returns in Module 14.